Frontend Theory Assignment - 3

Name: A. Sri Manikanta
University Roll No: 22515000013
Section: NC-05

1. What is the difference between frontend and backend JavaScript?

Frontend JavaScript is the code that runs in the user's browser. It is used to make a web page interactive, for example validating a form, showing or hiding things, handling button clicks and fetching data from an API. It can work with the DOM and browser objects like `window` and `localStorage`.

Backend JavaScript runs on a server, mostly using Node.js. It is used for things like handling requests, connecting to a database, login and authentication, and working with files. It cannot use the DOM or `window` because there is no browser, but it can use the file system and other server side modules.

So frontend JavaScript controls what the user sees in the browser, and backend JavaScript controls the logic and data on the server.

2. What is DOM?

DOM means Document Object Model. When a browser loads an HTML page, it converts the whole page into a tree of objects. Every tag, attribute and text in the page becomes a node in this tree. JavaScript can use the DOM to read the page and change it without reloading, such as changing text, styles or adding and removing elements.

```
document.getElementById("title").innerText = "Hello";
document.querySelector("p").style.color = "blue";
```

3. What is event in JavaScript?

An event is an action that happens on the web page, like clicking a button, pressing a key, submitting a form or loading the page. JavaScript can listen for these events and run a function when they happen. This is done using `addEventListener()`.

```
const btn = document.getElementById("btn");
btn.addEventListener("click", function () {
  alert("Button clicked");
});
```

Some common events are click, keydown, submit, mouseover, change and load.

4. What is event loop in JavaScript?

JavaScript is single threaded, which means it can run only one piece of code at a time. But it can still do tasks like timers and API calls without stopping the page. The event loop is what makes this possible.

Code runs in the call stack. Async tasks such as `setTimeout` and `fetch` are handled by the browser (Web APIs), and when they finish their callbacks wait in a queue. The event loop

keeps checking whether the call stack is empty. If it is empty, it takes the next callback from the queue and pushes it to the stack. Promise callbacks go in the microtask queue, which is checked before the normal callback queue.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// Output: A C B
```

B is printed last even though the delay is 0, because it has to wait until the main code finishes.

5. What is hoisting in JavaScript?

Hoisting means JavaScript moves the declarations of variables and functions to the top of their scope before running the code. Because of this we can sometimes use a function or variable before the line where it is declared.

Function declarations are completely hoisted, so they can be called before they are written. Variables declared with `var` are hoisted but their value is `undefined` until the assignment line. Variables declared with `let` and `const` are also hoisted but cannot be used before declaration, otherwise it gives a ReferenceError.

```
hello();                  // works
function hello() { console.log("Hi"); }

console.log(a);           // undefined
var a = 5;

console.log(b);           // ReferenceError
let b = 10;
```

6. What is event bubbling?

Event bubbling means that when an event happens on an element, it first runs on that element and then moves upward to its parent, then the parent's parent and so on, till the document. So the event bubbles up like a bubble in water.

```
<div id="parent">
  <button id="child">Click</button>
</div>

parent.addEventListener("click", () => console.log("parent"));
child.addEventListener("click", () => console.log("child"));
// clicking the button prints: child, parent
```

It can be stopped using `event.stopPropagation()`. Bubbling is useful for event delegation, where we put one listener on the parent to handle clicks of many child elements.

7. What is the difference between named function, anonymous function and arrow function?

Named function: a function which has a name. It can be called again and again using that name, and function declarations are hoisted.

```
function add(a, b) {
```

```
  return a + b;
}
```

Anonymous function: a function without a name. It is usually stored in a variable or given directly as a callback.

```
const add = function (a, b) {
  return a + b;
};
```

Arrow function: a shorter way of writing a function, introduced in ES6. It uses the => symbol. It does not have its own `this`, it takes `this` from the surrounding code, and it cannot be used as a constructor.

```
const add = (a, b) => a + b;
```

8. Explain for-of and for-in loop?

The for...in loop is used to go through the keys (property names) of an object.

```
const person = { name: "Sri", age: 20 };
for (let key in person) {
  console.log(key, person[key]);
}
// name Sri
// age 20
```

The for...of loop is used to go through the values of iterable things like arrays, strings, maps and sets. It does not work directly on normal objects.

```
const fruits = ["apple", "mango", "banana"];
for (let f of fruits) {
  console.log(f);
}
// apple mango banana
```

So for...in gives keys (or index in arrays) and for...of gives values.

9. Explain any five array methods?

1. push() and pop(): push() adds an element at the end of the array and pop() removes the last element.

```
let a = [1, 2];
a.push(3);    // [1, 2, 3]
a.pop();      // [1, 2]
```

2. map(): runs a function on every element and returns a new array.

```
[1, 2, 3].map(n => n * 2);     // [2, 4, 6]
```

3. filter(): returns a new array that has only the elements which satisfy a condition.

```
[1, 2, 3, 4].filter(n => n % 2 === 0);     // [2, 4]
```

4. reduce(): reduces the whole array to a single value, for example the sum of all numbers.

```
[1, 2, 3, 4].reduce((sum, n) => sum + n, 0);     // 10
```

5. find(): returns the first element that satisfies the condition. If nothing matches it returns undefined.

```
[5, 12, 8].find(n => n > 10);     // 12
```

10. What is callback method?

A callback is a function that is passed as an argument to another function and is called later, usually after some work is completed. In JavaScript functions can be passed like values, so this is possible. Callbacks are mostly used in asynchronous code such as timers, events and API requests.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}

greet("Sri", function () {
  console.log("Welcome");
});

setTimeout(function () {
  console.log("Runs after 2 seconds");
}, 2000);
```

If too many callbacks are nested inside one another the code becomes hard to read, which is called callback hell. Promises and async/await are used to avoid this.